TECHNICAL DOCUMENT

Trusted Integration

Update and Delete Log — Technical Document

Change tracking driven by a setup table, a log table, table-trigger event subscribers, a log-creation codeunit and a read/acknowledge API for 3rd-party synchronisation.

Item	Value
App	DailyOptimizer by Optimizers, version 28.0.0.13
Last updated	2026-10-20
Task / branch	DA1-T193 Trusted Integration (branch dev_ah)
Audience	BC developers who maintain the feature; 3rd-party integrators who consume the API
Verified in	Sandbox NL_Test (company CRONUS NL): Playwright against the live web client, automated tests (section 11.2) and live HTTP calls (section 11.3)
API namespace	BC365Optimizer / Planning / v1.0 (entity set UpdateAndDeleteLogs)

Status of verification

Sections 2 to 7 describe code that is published to NL_Test. Modify, Delete and Rename logging on Job, Job Task, Resource and Skill Code, and Modify logging on Day Planning, was exercised through the BC web client and, since 2026-10-20, by 7 automated tests (all passed, section 11.2). **Modify and Delete logging uses the database triggers (OnDatabaseModify / OnDatabaseDelete), not the global-trigger events**, because in BC 28.5 the global Modify/Delete events do not fire (section 5, section 13.5). Changes made by the 3rd party through the API/OData/SOAP endpoints are deliberately not logged. The log API (GET list) and the DayPlannings API were called over HTTP on NL_Test with an app-only token; the third-party skip is verified by those live calls only, because unit tests cannot run under the API client type (section 11.4). Section 12 lists the open points.

Contents

1. Purpose and scope
2. Concept and architecture
3. Object inventory
4. Setup table: API Record Exposed Opti (what must be filled in)
5. Event subscribers (codeunit 50627)
6. Log creation (codeunit 50605)
7. Log table: Update and Delete Log Opti
8. Log API for 3rd parties (page 50727)
9. Integration process (recommended sequence)
10. Postman walkthrough for 3rd parties
11. Worked examples and test evidence
12. Limitations, risks and open points
13. Operations: adding a table, deployment, repairs, pitfalls
14. Appendix: source files

1. Purpose and scope

A 3rd-party system has to mirror selected Business Central master data (Job, Job Task, Resource, Skill Code). Instead of the 3rd party scanning whole tables, Business Central records what changed in a log table and exposes that log through an API. The 3rd party reads the pending entries, applies them on its side and acknowledges each entry by setting its status to Synchronized.

The design goals were:

- **Setup-driven.** Which tables are tracked, and which of their fields matter, is decided by data (one row per API page), not by adding one event subscriber per table.
- **Only fields the 3rd party can see.** A change is logged only if the changed field is exposed on the registered API page.
- **No effect on untracked tables.** Tables without a setup row get no trigger and no extra code.
- **One homogeneous log shape.** Every row says what happened (Action), to which record (Record SystemId + EntitySetName), which API field (API Field Name) and the old and new value.
- **Acknowledgement through the API.** The consumer PATCHes Status; BC stamps the synchronisation time.
- **Changes made by the 3rd party are not logged.** A modify, delete or rename performed through the API, OData or SOAP endpoints comes from the 3rd party itself, which already has it; logging it would echo the change back. Inserts are never logged, whoever makes them. Manual changes in the web client, background sessions and job queue entries are logged.
- **Date-time values are UTC.** In the Old Value / New Value columns a DateTime field is written as ISO 8601 UTC (for example 2026-10-20T08:00:00.000Z), the same representation the API pages use (section 6.6).

Out of scope: logging of inserts (see section 11), transport/retry logic on the 3rd-party side, and any purge or archiving of the log.

2. Concept and architecture

2.1 Why the table triggers

Business Central offers two native ways to react to every table write without writing a subscriber per table: the Change Log feature and the table trigger events in codeunit `GlobalTriggerManagement` (the mechanism the Change Log, Dataverse integration and API webhooks use). The standard Change Log was not used because it writes into its own table, does not know about API pages (so it cannot restrict logging to exposed fields), and has no SystemId, entity set name or synchronisation status. The table trigger events give the same table-agnostic hook while keeping full control over what is written.

2.2 How the hook works

- BC asks, per table and per session, which triggers are wanted. Two events answer this question and codeunit 50627 subscribes to both: `OnAfterGetDatabaseTableTriggerSetup` (database triggers) and `OnAfterGetGlobalTableTriggerMask` (global triggers). Both are answered only when the table has a row in the setup table.
- **Modify and Delete** use the database triggers: the setup event switches on `OnDatabaseModify` and `OnDatabaseDelete` (not Insert, not Rename). BC then raises `OnBeforeOnDatabaseModify`(RecRef), `OnAfterOnDatabaseModify`(RecRef) and `OnAfterOnDatabaseDelete`(RecRef). The after-modify event carries no xRec, so codeunit 50627 captures the old record in the before-modify event (section 5.2).
- **Rename** still uses the global trigger: the mask event sets the bits for Modify, Delete and Rename (mask 14) and BC raises `OnAfterOnGlobalRename`(RecRef, xRecRef). RecRef holds the new values, xRecRef the previous ones.
- Codeunit 50627 forwards each event to codeunit 50605, which decides what to log and writes rows into table 50607.
- Before anything is logged, codeunit 50627 checks the client type; calls made by the 3rd party itself are skipped (section 5.1).

Trigger mask bits: 1 = Insert, 2 = Modify, 4 = Delete, 8 = Rename. Insert (bit 1) is deliberately never switched on by this code, and the database Insert and Rename triggers stay off. **Why two mechanisms:** in BC 28.5 the global events `OnAfterOnGlobalModify` and `OnAfterOnGlobalDelete` never fire; only `OnAfterOnGlobalRename` does (section 13.5). The mask still carries the Modify and Delete bits; they are harmless but no longer used.

2.3 Data flow



3. Object inventory

Type	ID	Name	Role
Table	50614	API Record Exposed Opti	Setup: which API pages (and therefore tables) are tracked
Page	50639	API Record Exposed Opti	List page to maintain the setup rows (Administration)
Table	50607	Update and Delete Log Opti	The log; one row per logged change
Page	50726	Update and Delete Log List	Read-only list of the log for BC users
Enum	50624	Update and Delete Log Status	Pending (0), Synchronized (10)
Enum	50625	Update and Delete Log Action	Modify (0), Delete (1), Rename (2). There is no Insert action.
Codeunit	50627	API Record Exposed Triggers	Event subscribers on GlobalTriggerManagement: switch the database triggers (Modify, Delete) and the global Rename trigger on, capture the old record, skip 3rd-party client types, forward events
Codeunit	50605	API Record Exposed Mgt.	Builds and inserts the log rows (FormatValue helper formats DateTime as UTC)
API page	50727	UpdateAndDeleteLogApi Opt	3rd-party endpoint: read the log, PATCH status
API pages (tracked)	50603, 50605, 50606, 50611, 50615, 50649, 50658, 50659, 50671, 50672	JobApi Opt, JobTaskApi Opt, DayPlanningApi Opt, Resource API Optimization, Resource Group API Opt., ApiProjectTypeOpt, CustomersOpt, VendorOpt, SkillCodeApi Opt, ResourceSkillApi Opt	Currently registered in the setup table - 10 rows (tables Job 167, Job Task 1001, Day Planning 50610, Resource 156, Resource Group 152, Project Type Opt. 50602, Customer 18, Vendor 23, Skill Code 5955, Resource Skill 5956)

4. Setup table: API Record Exposed Opti (what must be filled in)

The setup table is the only thing an administrator maintains. One row per API page that should be tracked.

Field	Type	Filled by	Description
API Page No. (1)	Integer, primary key	User	Lookup on Page Metadata filtered to PageType = API. Required (NotBlank).
Table ID (2)	Integer, not editable	System (OnValidate of API Page No.)	Set to the SourceTable of the chosen page. A page that does not exist raises an error.
API Page Name (10)	FlowField (Text 100)	System	Lookup of Page Metadata.Name for the page.
Table Name (20)	FlowField (Text 100)	System	Lookup of Table Metadata.Name for Table ID.

4.1 Currently registered rows

API Page No.	API Page Name	Table ID	Table Name	EntitySetName
50603	JobApi Opt	167	Job	Jobs

API Page No.	API Page Name	Table ID	Table Name	EntitySetName
50605	JobTaskApi Opt	1001	Job Task	jobTasks
50606	DayPlanningApi Opt	50610	Day Planning	DayPlannings
50611	Resource API Optimization	156	Resource	Resources
50615	Resource Group API Opt.	152	Resource Group	ResourceGroups
50649	ApiProjectTypeOpt	50602	Project Type Opt.	projectTypes
50658	CustomersOpt	18	Customer	Customers
50659	VendorOpt	23	Vendor	Vendors
50671	SkillCodeApi Opt	5955	Skill Code	SkillCodes
50672	ResourceSkillApi Opt	5956	Resource Skill	ResourceSkills

The setup table in NL_Test holds these 10 rows (verified on page 50639). The log is written for every registered table. The API returns all of it; a 3rd party that needs only some entities filters on entitySetName, for example `$filter=status eq 'Pending' and entitySetName eq 'Resources'`.

4.2 Rules the setup drives

- A table is tracked if any setup row has its Table ID. If two API pages share a table, every change is logged once per page (each row carries that page's EntitySetName).
- Which fields are tracked is decided by the API page itself: a field change is logged only if the page has a control bound to that field (see 6.2). Adding a field to the API page starts tracking it; removing it stops tracking.
- Setup changes take effect for new BC sessions only, because the trigger setup is cached per session (section 12).
- Deleting a setup row stops tracking after the same session refresh. Existing log rows are kept.

5. Event subscribers (codeunit 50627)

Codeunit 50627 API Record Exposed Triggers contains the only event subscribers. They are generic: adding a tracked table never means adding a subscriber. The codeunit is `SingleInstance` because it holds the old record between the before-modify and after-modify events (section 5.2). All events are raised by codeunit `GlobalTriggerManagement`.

Procedure	Event	What it does
EnableDatabaseTriggers	OnAfterGetDatabaseTableTriggerSetup(TableId, var OnDatabaseInsert, var OnDatabaseModify, var OnDatabaseDelete, var OnDatabaseRename)	If the setup table has a row for TableId, sets OnDatabaseModify and OnDatabaseDelete to true. Insert and Rename stay off.
CaptureOldRecord (+ ReadOldRecord, a TryFunction)	OnBeforeOnDatabaseModify(RecRef)	Skips 3rd-party calls. Otherwise re-reads the stored record from the database into a RecordRef held in the SingleInstance codeunit and remembers its table number and SystemId. The read is a TryFunction, so a failure can never break the user's save.
LogModify	OnAfterOnDatabaseModify(RecRef)	Skips 3rd-party calls. If the captured old record belongs to this exact table and SystemId, clears the slot and calls ModifiedFieldLog(OldRecRef, RecRef, RecRef.Number()). Otherwise exits without logging.
LogDelete	OnAfterOnDatabaseDelete(RecRef)	Skips 3rd-party calls. Calls DeletedRecordLog(RecRef).

Procedure	Event	What it does
EnableForRegisteredTables	OnAfterGetGlobalTableTriggerMask(TableID, var TableTriggerMask)	If the setup table has a row for TableID, sets the Modify (2), Delete (4) and Rename (8) bits and keeps the Insert bit and higher bits as other subscribers set them: TableTriggerMask := (TableTriggerMask div 16) * 16 + 14 + (TableTriggerMask mod 2). Needed for Rename.
LogRename	OnAfterOnGlobalRename(RecRef, xRecRef)	Skips 3rd-party calls. Calls RenamedRecordLog(RecRef, xRecRef).
IsThirdPartyCall (local)	none (helper)	CurrentClientType() in [ClientType::Api, ClientType::ODataV4, ClientType::SOAP].

```
[EventSubscriber(ObjectType::Codeunit, Codeunit::GlobalTriggerManagement,
                 'OnAfterGetDatabaseTableTriggerSetup', '', false, false)]
[InherentPermissions(PermissionObjectType::TableData, Database::"API Record Exposed Opti", 'R')]
local procedure EnableDatabaseTriggers(TableId: Integer; var OnDatabaseInsert: Boolean;
    var OnDatabaseModify: Boolean; var OnDatabaseDelete: Boolean; var OnDatabaseRename: Boolean)
var
    APIRecordExposed: Record "API Record Exposed Opti";
begin
    APIRecordExposed.SetRange("Table ID", TableId);
    if not APIRecordExposed.IsEmpty() then begin
        OnDatabaseModify := true;
        OnDatabaseDelete := true;
    end;
end;
```

```
local procedure LogModify(RecRef: RecordRef)      // OnAfterOnDatabaseModify
begin
    if IsThirdPartyCall() then exit;
    SystemId := RecRef.Field(RecRef.SystemIdNo()).Value();
    // only log when the stored old record belongs to this exact record
    if (OldTableNo <> RecRef.Number()) or (OldSystemId <> SystemId) then exit;
    Clear(OldTableNo); Clear(OldSystemId);
    APIRecordExposedMgt.ModifiedFieldLog(OldRecRef, RecRef, RecRef.Number());
end;
```

For the Rename mask the same arithmetic as before applies. AL has no bitwise OR on integers, so the mask is rebuilt: div 16 * 16 keeps all bits above the four trigger bits, + 14 turns on 2 + 4 + 8, and mod 2 preserves an Insert bit that another app may have set.

5.1 Changes made by the 3rd party are not logged

Every logging subscriber (CaptureOldRecord, LogModify, LogDelete, LogRename) first calls IsThirdPartyCall(): if `CurrentClientType()` is Api, ODataV4 or SOAP the event is ignored. The 3rd party that PATCHes, DELETEs or renames a record through the API already has that change, so logging it would only echo it back as Pending work. Everything else is logged: the web client (manual changes by users), background sessions and job queue entries. Inserts are never logged, including a record created by an API POST.

> **Consequence for cascades and side effects**
>
> The skip is decided by the client type of the session, not by who typed the value. AL code that runs inside an API request (a cascade delete, a field recalculated by a trigger, a related record modified by an OnModify) is therefore not logged either. The 3rd party must re-read dependent records after its own API call (section 12, item 12).

5.2 Capturing the old record, and its known limit

The database after-modify event has no xRec. `CaptureOldRecord` therefore re-reads the record from the database (the stored, not yet overwritten version) into `OldRecRef` in the before-modify event and stores the table number and SystemId as a key. `LogModify` compares that key with the record in the after-modify event and logs only on a match. The read is a TryFunction, so a failure leaves the key empty and the change is simply not logged instead of breaking the user's save.

> **Known limit: one old-record slot**
>
> There is one slot only. If another record of a registered table is modified inside a Modify trigger of the first record, the nested before-modify event overwrites the slot (and the nested after-modify event clears it). When the outer after-modify event then fires, the key no longer matches and the outer change is **skipped rather than logged with wrong old values**. This is a deliberate trade-off: a missing row is detectable, a wrong row is not.

5.3 When the triggers run

Database triggers fire regardless of the RunTrigger flag, so a Modify(false) or Delete(false) on a registered table is logged too (the earlier design, which used the global Modify/Delete events, missed those). Rename is still raised through the global trigger; its behaviour with Rename(false) was not tested.

6. Log creation (codeunit 50605)

Codeunit 50605 API Record Exposed Mgt. turns an event into log rows. Its three public procedures are called by codeunit 50627; InsertLog is the single place that writes to table 50607.

Procedure	Called for	Result
ModifiedFieldLog(OldRec, NewRec, TableID)	Modify	One Modify row per changed field that is exposed on a registered API page.
DeletedRecordLog(RecRef)	Delete	One Delete row per registered API page of the table (no field).
RenamedRecordLog(RecRef, xRecRef)	Rename	One Rename row per primary-key field, per registered API page.
GetApiFieldName (local)	helper	Looks up the API control name of a field on an API page (Page Control Field).
FormatValue (local)	helper	Text of a field value for Old/New Value: DateTime fields as ISO 8601 UTC (Format(value, 0, 9)), all other types with Format().
InsertLog (local)	helper	Fills and inserts one row; Status = Pending, Modified At = CurrentDateTime().

6.1 Common guards

- Temporary records are ignored (RecRef.IsTemporary()).
- A registered API page that no longer exists is skipped (if PageMetadata.Get(...)) so a stale setup row can never make a user's save fail.
- Inherent permissions are declared on the public procedures: read on the setup table, read and insert on the log table. Ordinary users therefore do not need direct permissions on these tables for their own save to succeed. The system metadata tables (Page Metadata, Page Control Field) cannot be listed in an InherentPermissions attribute (compiler error AL0720) and are assumed readable by every user. This was not tested with a non-admin user.

- Entry No. is AutoIncrement, so concurrent saves cannot produce duplicate entry numbers or serialise on a FindLast.
- The 3rd-party client-type check and the old-record capture happen earlier, in codeunit 50627 (section 5).

6.2 Modify: which changes are logged

For every setup row of the table, the code loops over all fields of the record. For a normal (non-FlowField) field whose formatted value differs between old and new, it looks up the field on the API page. If the page has a control bound to that field, one row is written; otherwise the change is ignored. Because the comparison is per field and the event is raised once per Modify, exactly one row per changed field is written (no duplicates; verified by test ManualModifyLogsOncePerField, section 11.2).

```
APIRecordExposed.SetRange("Table ID", TableID);
if APIRecordExposed.FindSet() then
    repeat
        if PageMetadata.Get(APIRecordExposed."API Page No.") then
            for i := 1 to NewRecRef.FieldCount() do begin
                NewFldRef := NewRecRef.FieldIndex(i);
                if NewFldRef.Class() = FieldClass::Normal then begin
                    OldFldRef := OldRecRef.Field(NewFldRef.Number());
                    if FormatValue(OldFldRef) <> FormatValue(NewFldRef) then
                        if GetApiFieldName(APIRecordExposed."API Page No.", TableID,
                                           NewFldRef.Number(), ControlName) then
                            InsertLog(NewRecRef, PageMetadata.EntitySetName, ControlName,
                                      Action::Modify, NewFldRef.Number(),
                                      FormatValue(OldFldRef), FormatValue(NewFldRef));
                end;
            end;
    until APIRecordExposed.Next() = 0;
```

The API field name is the control name that the API page declares (for example description, no_, billtoCustomerNo), read from the metadata table Page Control Field (PageNo, TableNo, FieldNo -> ControlName). It is exactly the property name a 3rd party sees in the entity's JSON.

6.3 Delete

A deleted record produces one row per registered page: Action = Delete, Field No. = 0, API Field Name blank, Old Value and New Value blank. The Record SystemId identifies what to remove on the 3rd-party side; the record no longer exists in BC.

6.4 Rename (single and composite primary keys)

A rename changes primary-key fields. The log keeps every Rename row in the same shape, one row per primary-key field:

- Single-field key (Resource No., Skill Code, Job No.): one row. API Field Name = the API name of the key field (blank if the page does not expose it), Field No. = the key field, Old/New Value = the plain key values.
- Composite key (Job Task: Job No. + Job Task No.): n rows, one for each key field. A key field that did not change is still written, with identical Old and New Value, so the 3rd party always receives the complete key.

```
KeyRef := RecRef.KeyIndex(1);                       // primary key
...
for i := 1 to KeyRef.FieldCount() do begin
    FldRef := KeyRef.FieldIndex(i);                 // new value
    xFldRef := xRecRef.Field(FldRef.Number());      // old value
    if not GetApiFieldName(APIPageNo, RecRef.Number(), FldRef.Number(), ControlName) then
        ControlName := '';
    InsertLog(RecRef, PageMetadata.EntitySetName, ControlName, Action::Rename,
              FldRef.Number(), FormatValue(xFldRef), FormatValue(FldRef));
end;
```

6.5 Field-by-field mapping of a log row (InsertLog)

Log field	Value written
Entry No.	Assigned by AutoIncrement
Action	Modify / Delete / Rename
Record SystemId	RecRef.Field(RecRef.SystemIdNo()).Value() - the SystemId of the changed record (stays the same across a rename)
EntitySetName	EntitySetName of the registered API page (Jobs, jobTasks, Resources, SkillCodes ...)
API Field Name	API control name of the changed/key field; blank for Delete, and for a Rename key field that is not exposed
Table No. / Field No.	Source table number; field number of the changed or key field (0 for Delete)
Old Value / New Value	FormatValue() of the old and new value (DateTime as ISO 8601 UTC, see 6.6), truncated to 100 characters; blank for Delete
Modified At	CurrentDateTime() at logging time
Status	Pending
Synchronized At	Left blank; set by the API when Status becomes Synchronized

6.6 Date-time values are UTC

For Modify and Rename logging the helper `FormatValue` checks the field type. A DateTime field is written as `Format(value, 0, 9)`, the XML/ISO 8601 format in UTC, for example `2026-10-20T08:00:00.000Z` (always ending in Z). Without this, Format() would write the value in the BC session's local time zone, and a 3rd party could not tell which zone it was in. Every other field type is written with plain Format().

- The date-time fields that the API pages expose are serialised by BC as UTC as well: systemCreatedAt, systemModifiedAt, Modified At / Synchronized At on the log API, and the project location created/modified fields. A 3rd party can therefore compare logged old/new values with values it reads from the API without any conversion.
- Log rows written before this change keep their old local-time values; they are not rewritten.

7. Log table: Update and Delete Log Opti (table 50607)

No.	Field	Type	Notes
1	Entry No.	Integer	Primary key, AutoIncrement. Ordering key for consumers.
5	Action	Enum 50625	Modify = 0, Delete = 1, Rename = 2

No.	Field	Type	Notes
10	Record SystemId	Guid	SystemId of the record that changed
18	EntitySetName	Text[100]	OData entity set of the registered API page
20	Table No.	Integer	Source table (167 Job, 1001 Job Task, 156 Resource, 5955 Skill Code ...)
21	Field No.	Integer	Changed/key field number; 0 for Delete
30	Old Value	Text[100]	Previous value (blank on first fill of an empty field and on Delete). DateTime: ISO 8601 UTC.
31	New Value	Text[100]	New value (blank on Delete). DateTime: ISO 8601 UTC.
40	Modified At	DateTime	When the change was logged
50	Synchronized At	DateTime	Set when Status becomes Synchronized, cleared if it returns to Pending
60	Status	Enum 50624	Pending = 0, Synchronized = 10
70	API Field Name	Text[150]	API control name (see 6.5)

Page 50726 (Update and Delete Log List) shows the table read-only, newest entry first, with all of the above except Synchronized-At editing. There is no BC-side action to delete or edit rows; corrections to existing data are done with one-off repair code (section 13.3).

8. Log API for 3rd parties (page 50727)

8.1 Endpoint

```
GET https://api.businesscentral.dynamics.com/v2.0/{{company_name}}/api/
    BC365Optimizer/Planning/v1.0/companies({{CompVar}})/UpdateAndDeleteLogs
```

- {{company_name}} = the environment segment of the URL (for example NL_Test), as used in the existing Postman collection (the form v2.0/{tenantId}/{environment} also works)
- {{CompVar}} = GUID of the company
- Authentication: OAuth 2.0 (Microsoft Entra ID) like every BC API.
- The API user needs read access plus modify on table 50607 (see 8.5).

8.2 Properties

Property	Value
PageType / Publisher / Group / Version	API / BC365Optimizer / Planning / v1.0
EntityName / EntitySetName	UpdateAndDeleteLog / UpdateAndDeleteLogs
Source table	Update and Delete Log Opti (50607)
OData key	SystemId (exposed as property id)
Insert / Delete	Not allowed. The log is written only by BC.
Writable property	status only. Every other property is read-only.

8.3 Properties of the entity

The property names follow the 3rd party's own table layout (columns id, systemId, fieldName, oldvalue, newvalue, update_at, date_time_sync, status).

JSON property	Source	Type / values	Writable
id	Log row SystemId	GUID - key of the log row	No
systemId	Record SystemId	GUID - the changed record	No
entryNo	Entry No.	Integer - ordering	No
action	Action	Modify | Delete | Rename	No
entitySetName	EntitySetName	Text - where to fetch/update the record (Jobs, jobTasks, Resources, SkillCodes)	No
fieldName	API Field Name	Text - API property name; blank for Delete and for non-exposed Rename keys	No
oldvalue	Old Value	Text (max 100); DateTime values as ISO 8601 UTC	No
newvalue	New Value	Text (max 100); DateTime values as ISO 8601 UTC	No
update_at	Modified At	DateTime (UTC in OData)	No
date_time_sync	Synchronized At	DateTime (UTC), null until synchronized	No
status	Status	Pending | Synchronized	Yes

The attached 3rd-party sample shows fieldName as dayplanning.description and status as lower-case pending. By decision the BC value is the bare API field name (description); the entity is carried separately in entitySetName. Status is returned with the enum member names Pending / Synchronized; the consumer should compare case-insensitively.

8.4 Typical calls

```
# 1. Pending work, oldest first
GET .../UpdateAndDeleteLogs?$filter=status eq 'Pending'&$orderby=entryNo asc&$top=100

# 2. Acknowledge one entry
PATCH .../UpdateAndDeleteLogs({id})
If-Match: *
Content-Type: application/json
{ "status": "Synchronized" }
-> BC sets date_time_sync = now. PATCHing back to "Pending" clears date_time_sync.

# 3. Fetch the live record behind an entry (not for Delete)
GET .../{entitySetName}({systemId})
    e.g. .../SkillCodes(e93872d1-f05e-f111-8e24-7ced8d3f3ccf)
```

Example response row (a Modify on a Skill Code description):

```
{
  "id": "...", "systemId": "e93872d1-f05e-f111-8e24-7ced8d3f3ccf", "entryNo": 12,
  "action": "Modify", "entitySetName": "SkillCodes", "fieldName": "description",
  "oldvalue": "Sanitair", "newvalue": "Sanitair LOGTEST",
  "update_at": "2026-10-08T05:13:00Z", "date_time_sync": null, "status": "Pending"
}
```

8.5 Server-side behaviour and permissions

- OnModifyRecord keeps Synchronized At in step with Status: moving to Synchronized stamps CurrentDateTime(); moving back to Pending clears it.
- The service principal / user calling the API needs permissions on table 50607 (read, modify) and execute on the API page. The existing Demolition permission set does not list the newer tables (50607, 50614); add them if a set is used for the integration user.
- The log API is written to by BC only. A PATCH of status by the 3rd party is a change on table 50607, which is not a registered (tracked) table, so it is not itself logged.

9. Integration process (recommended sequence)

Step	3rd party	Business Central
1	Poll the log: Status = Pending, ordered by entryNo ascending, paged.	Returns the rows as written by the triggers.
2	Group rows by systemId and action; process in entryNo order.	-
3	Modify: apply newvalue to the property named in fieldName on the record identified by systemId. A row whose oldvalue is blank means the field was filled for the first time (typically right after the record was created).	-
4	Rename: the rows of one rename share systemId and the same Modified At. Apply every row; for a composite key all key fields arrive (changed or not). The systemId does not change, so match on it, not on the key.	-
5	Delete: remove the record identified by systemId (it no longer exists in BC; fieldName and values are blank).	-
6	Optionally GET entitySetName(systemId) to read the full current record instead of replaying single field changes (not possible for Delete).	Serves the record through its own API page.
7	PATCH status = Synchronized for each processed id (or only after a whole batch succeeded).	Stamps date_time_sync.
8	On any failure leave the row Pending; it will be returned again on the next poll.	-

9.1 Idempotency and ordering

- Entry No. is strictly increasing, so a consumer can also keep the highest processed entryNo as a watermark in addition to the status.
- Applying a Modify twice is harmless (same newvalue). Applying a Delete to a record that is already gone must be tolerated.
- The log holds changes, not snapshots: process in order, or fetch the live record, never apply an older entry after a newer one for the same systemId and fieldName.
- Date-time old/new values are ISO 8601 UTC (ending in Z); the 3rd party can compare them with API values directly.

9.2 What the log does not contain: changes the 3rd party made itself

BC logs only changes that originate inside BC (web client, background sessions, job queue). A modify, delete or rename that the 3rd party sends through the API, OData or SOAP endpoints is **not** logged: the 3rd party already has that change, and logging it would make it re-apply its own edit. Inserts are never logged. For the integration this means:

- There is no echo to filter out: every Pending row is a change the 3rd party has not seen yet.

- Side effects of the 3rd party's own API call (a cascade delete, a field recalculated by an OnModify trigger, a related record changed by AL code) happen in an API session and are not logged either. After its own PATCH or DELETE, the 3rd party should GET the affected record and its dependent records if it needs the resulting values.

10. Postman walkthrough for 3rd parties

The log was designed so that a 3rd party can turn every entry mechanically into one API call on the entity it keeps in sync. The entity set (entitySetName), the record (systemId) and the property to change (fieldName, the API property name) are all in the log row, so no mapping table is needed. The target system is any API that offers the same entity sets and property names as the BC API pages (for example another instance of the same app). The JSON below is illustrative: the values come from real NL_Test log rows, but the exact HTTP response envelope of the target calls was not captured (section 12, item 1).

10.1 The rule: log row -> API request

Log action	Request on the target API	Body
Modify	PATCH {entitySetName}({systemId})	{ "{fieldName}": "{newvalue}" } e.g. Resources, name: { "name": "Test resource B" }
Rename (single key)	PATCH {entitySetName}({systemId})	{ "{fieldName}": "{newvalue}" } e.g. Resources, no_: { "no_": "TESTRES4" }
Rename (composite key)	ONE PATCH {entitySetName}({systemId}) for all rows of the rename	{ "{fieldName1}": "{newvalue1}", "{fieldName2}": "{newvalue2}" } e.g. { "jobNo_": "TESTJOB4", "jobTaskNo_": "T2" }
Delete	DELETE {entitySetName}({systemId})	none

After the call succeeds the 3rd party acknowledges the log row in BC (Request 3 below). Only that acknowledgement changes data in BC's log; the PATCH/DELETE goes to the system being synchronised. If the target of the PATCH/DELETE is itself a BC API, that BC does not log the call either (section 5.1).

How the 3rd party uses each property of a log row

Property	Used for
action	Chooses the request: Modify and Rename -> PATCH, Delete -> DELETE
entitySetName	The entity set segment of the target URL (Resources, SkillCodes, Jobs, jobTasks)
systemId	The record key in the target URL: entitySetName(systemId). Stable across renames.
fieldName	The JSON property name in the PATCH body (blank for Delete)
newvalue	The JSON property value in the PATCH body (date-times are ISO 8601 UTC)
oldvalue	Optional conflict check against the current target value; blank means the field was filled for the first time
update_at	Groups the rows of one composite-key rename together with systemId and action
entryNo	Processing order (ascending) and optional watermark
status / id	Filter status eq 'Pending' to find work; PATCH status = Synchronized on the row id to acknowledge it

10.2 Postman environment

Variable	Example / meaning
company_name	The environment segment used in the URL (for example NL_Test, Production). Same variable name as in the existing Postman collection.
CompVar	GUID of the company. Get it once with GET https://api.businesscentral.dynamics.com/v2.0/{{company_name}}/api/v2.0/companies
tenantId	Microsoft Entra tenant ID (only needed for the token request, Request 0)
bcBaseUrl	https://api.businesscentral.dynamics.com/v2.0/{{company_name}}/api/BC365Optimizer/Planning/v1.0/companies({{CompVar}}) - the BC that owns the log; entity sets are appended, for example {{bcBaseUrl}}/resources
targetBaseUrl	Base URL (including company) of the API that is being synchronised, with the same entity sets and property names. In the simplest case it is the same URL shape as bcBaseUrl.
clientId / clientSecret	Entra app registration that is registered in BC (Microsoft Entra Applications) and has a permission set including table 50607
token	Bearer token for BC, filled by Request 0
logId, systemId, entitySetName, fieldName, newvalue	Filled by the Tests script of Request 1

10.3 Request 0 - get a BC access token

```
POST https://login.microsoftonline.com/{{tenantId}}/oauth2/v2.0/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
client_id={{clientId}}
client_secret={{clientSecret}}
scope=https://api.businesscentral.dynamics.com/.default

Tests tab:
pm.environment.set("token", pm.response.json().access_token);
```

On the BC requests use Authorization = Bearer Token {{token}} and the header Accept: application/json.

10.4 Request 1 - list the pending log entries (BC)

```
GET {{bcBaseUrl}}/UpdateAndDeleteLogs
    ?$filter=status eq 'Pending'&$orderby=entryNo asc&$top=100

i.e. https://api.businesscentral.dynamics.com/v2.0/{{company_name}}/api/
     BC365Optimizer/Planning/v1.0/companies({{CompVar}})/UpdateAndDeleteLogs?$filter=...
```

Put $filter, $orderby and $top in the Params tab. The rows are in value; when more exist the response has @odata.nextLink, called as is. Tests tab - take the first pending row into variables for the next request:

```
const rows = pm.response.json().value;
if (rows.length) {
   const r = rows[0];
   pm.environment.set("logId", r.id);
   pm.environment.set("systemId", r.systemId);
   pm.environment.set("entitySetName", r.entitySetName);
   pm.environment.set("fieldName", r.fieldName);
   pm.environment.set("newvalue", r.newvalue);
}
```

10.5 Scenario A - Modify

Example: a Resource name was changed in BC. Request 1 returns:

```
{ "id": "a1b2...", "systemId": "6bb55607-5e7b-f111-8070-70a8a52cf9b7", "entryNo": 14,
  "action": "Modify", "entitySetName": "Resources", "fieldName": "name",
  "oldvalue": "Dianne Buchanan", "newvalue": "Dianne Buchanan LOGTEST",
  "update_at": "2026-10-08T05:15:00Z", "date_time_sync": null, "status": "Pending" }
```

Rule: if the log says Modify on Resources field name, the 3rd party must PATCH the Resources entity of the target API with the value taken from the log (newvalue):

Request 2 - PATCH the target entity

```
General form: PATCH {{targetBaseUrl}}/{{entitySetName}}({{systemId}})
              body { "{{fieldName}}": "{{newvalue}}" }
```

For this row, on the Resources endpoint (GET .../companies({{CompVar}})/resources returns the same entity):

```
PATCH https://api.businesscentral.dynamics.com/v2.0/{{company_name}}/api/
      BC365Optimizer/Planning/v1.0/companies({{CompVar}})/
      resources(6bb55607-5e7b-f111-8070-70a8a52cf9b7)
If-Match: *
Content-Type: application/json

{ "name": "Dianne Buchanan LOGTEST" }
```

- The property name is always fieldName; the value is always newvalue. In Postman build the body as { "{{fieldName}}": "{{newvalue}}" } with the variables from Request 1.
- oldvalue is only for the 3rd party's own conflict check (compare it with the current target value if needed).
- If oldvalue is blank the field was filled for the first time, typically right after the record was created in BC. If the target answers 404 because the record does not exist yet, create it first (POST to the entity set, then repeat the PATCH); BC does not log inserts (section 12, item 2).
- The entity can be read with GET {{bcBaseUrl}}/resources (all) or GET {{bcBaseUrl}}/resources({{systemId}}) (one), for example to check the current value before or after the PATCH. Entity set names in the URL are not case sensitive.
- Other examples: Skill Code description: PATCH SkillCodes(id) { "description": "..." }; Job billtoCustomerNo: PATCH Jobs(id) { "billtoCustomerNo": "10000" }.

Request 3 - acknowledge the log entry (BC)

```
PATCH {{bcBaseUrl}}/UpdateAndDeleteLogs({{logId}})
If-Match: *
Content-Type: application/json

{ "status": "Synchronized" }
```

A successful call returns the row with status = Synchronized and date_time_sync set by BC; the row is no longer returned by Request 1. If-Match: * is required for PATCH. Send Request 3 only after Request 2 succeeded.

10.6 Scenario B - Rename

A rename changes the key of a record, but the record keeps its systemId, so the target entity is addressed by systemId, never by the old key.

B1 - single-field key (Resource, Skill Code, Job)

```
{ "systemId": "10ceafa3-dbc2-f111-a056-7c1e52874287", "entryNo": 20, "action": "Rename",
  "entitySetName": "Resources", "fieldName": "no_", "oldvalue": "TESTRES3",
  "newvalue": "TESTRES4", ... }

PATCH {{targetBaseUrl}}/Resources(10ceafa3-dbc2-f111-a056-7c1e52874287)
If-Match: *
{ "no_": "TESTRES4" }
```

Then acknowledge with Request 3.

B2 - composite key (Job Task: Job No. + Job Task No.)

One rename produces one log row per key field, all with the same systemId and update_at:

```
{ "entryNo": 43, "action": "Rename", "entitySetName": "jobTasks", "fieldName": "jobNo_",
  "oldvalue": "TESTJOB4", "newvalue": "TESTJOB4",
  "systemId": "47a5d87f-efc2-f111-a056-7c1e52874287", ... }
{ "entryNo": 44, "action": "Rename", "entitySetName": "jobTasks", "fieldName": "jobTaskNo_",
  "oldvalue": "T1", "newvalue": "T2",
  "systemId": "47a5d87f-efc2-f111-a056-7c1e52874287", ... }

PATCH {{targetBaseUrl}}/jobTasks(47a5d87f-efc2-f111-a056-7c1e52874287)
If-Match: *
{ "jobNo_": "TESTJOB4", "jobTaskNo_": "T2" }
```

- Group the pending rows by systemId + update_at + action and send one PATCH whose body has one property per row.
- A key field that did not change still arrives (old = new) so the 3rd party always has the complete key.
- Acknowledge every id of the group with Request 3 (here entries 43 and 44).
- If fieldName is blank on a Rename row, that key field is not exposed on the API page: leave it out of the body, but still acknowledge the row.

10.7 Scenario C - Delete

```
{ "id": "...", "systemId": "dbf2c5de-dbc2-f111-a056-7c1e52874287", "entryNo": 45,
  "action": "Delete", "entitySetName": "Jobs", "fieldName": "", "oldvalue": "",
  "newvalue": "", "status": "Pending", ... }

DELETE {{targetBaseUrl}}/Jobs(dbf2c5de-dbc2-f111-a056-7c1e52874287)
If-Match: *
```

- No body. fieldName, oldvalue and newvalue are blank by design.
- Do not read the entity in BC for a Delete: the record no longer exists there and BC answers 404.
- If the target answers 404 the record is already gone: treat as success and acknowledge.
- Deleting a parent in BC removes its children there, but the children are currently not logged (section 12, item 3). When a parent Delete arrives, also delete its children on the target (for example the Job Tasks of a deleted Job).
- Then acknowledge with Request 3.

10.8 Collection layout and ordering rules

Folder / request	Purpose
0 Auth - Get token	Request 0
1 Log - List pending	Request 1 (BC): $filter, $orderby=entryNo asc, $top
2 Target - Apply	Request 2 on the target API: PATCH (Modify, Rename) or DELETE, built from the log variables
3 Log - Acknowledge	Request 3 (BC): PATCH status = Synchronized

- Process rows in entryNo ascending order; never apply an older entry after a newer one for the same systemId and fieldName.
- Acknowledge only what was applied successfully. A failed row stays Pending and is returned again by the next list call.
- Run the folders with the Postman Collection Runner (or a monitor) on a schedule; the Tests script of Request 1 feeds Requests 2 and 3.
- A watermark of the highest acknowledged entryNo is an optional extra safety net.

10.9 Typical HTTP results

Status	Meaning	What to do
200 / 204	GET / PATCH / DELETE succeeded	Continue
400	Bad request, for example an invalid status value or property name	Use Pending or Synchronized exactly; use fieldName as delivered
401	Token missing, expired or wrong audience	Repeat Request 0; scope must be https://api.businesscentral.dynamics.com/.default
403	No permission on table 50607 or the API page	Add the permission set (section 8.5)
404	Record not found (deleted) or wrong company/environment	Expected for a repeated Delete; otherwise check companyId and environment
412 / 428	If-Match header missing or not matching	Send If-Match: * on PATCH and DELETE
429 / 5xx	Throttled or temporary server error	Retry later with back-off; the entry stays Pending

11. Worked examples and test evidence

The rows below were produced in NL_Test through the BC web client and read back from page 50726. Records named TEST* were created and deleted only for testing. The automated tests (11.2) and the live HTTP calls (11.3) were added on 2026-10-20.

11.1 Manual evidence from the web client

Entry	Action	Entity set	Table	Field	API field	Old value	New value
12	Modify	SkillCodes	5955	2	description	Sanitair	Sanitair LOGTEST
14	Modify	Resources	156	2	name	Dianne Buchanan	Dianne Buchanan LOGTEST
16	Modify	Jobs	167	3	description	Riverside Office Tower	Riverside Office Tower LOGTEST
27	Modify	Jobs	167	5	billtoCustomerNo	(blank)	10000
51	Modify	DayPlannings	50610	13	startTimeRequested	09:00:00	10:00:00
52	Modify	DayPlannings	50610	14	endTimeRequested	17:00:00	18:00:00
20	Rename	Resources	156	1	no_	TESTRES3	TESTRES4
24	Rename	SkillCodes	5955	1	code	TESTSKL1	TESTSKL2
29	Rename	Jobs	167	1	no_	TESTJOB1	TESTJOB2
43	Rename	jobTasks	1001	1	jobNo_	TESTJOB4	TESTJOB4
44	Rename	jobTasks	1001	2	jobTaskNo_	T1	T2
21 / 25 / 45	Delete	Resources / SkillCodes / Jobs	156 / 5955 / 167	0	(blank)	(blank)	(blank)

Behaviours confirmed

- Fields that are not exposed on the API page are not logged. Example: changing Unit Price (and the derived Profit %) on a Resource produced no log row, while changing Name did.
- Creating a record is not logged (Insert trigger off); the first values typed afterwards show up as Modify rows with a blank old value (entries 18, 22, 26, 31).
- Rename and Delete both work for single-key tables (Resource, Skill Code, Job); a composite-key rename (Job Task) produces one row per key field (entries 43 and 44).
- Entry numbers increase continuously (AutoIncrement), including after the earlier log rows were deleted by a repair run.
- The API Field Name equals the control name of the API page (for example no_, jobTaskNo_).

- The manual rows above were written by the earlier implementation (global-trigger events for Modify and Delete). Since the switch to the database triggers (section 5) the behaviour is re-verified by the automated tests in 11.2.

11.2 Automated tests (separate test app)

The tests live in a separate test app, kept out of the DailyOptimizer app: folder demolition_tests, app DailyOptimizerTests 1.0.0.2, codeunit 50900 "API Rec Exposed Log Test". They were run on NL_Test (company CRONUS NL) on 2026-10-20 with the al_run_tests tool. **Result: 7 tests passed, 0 failed (8 result lines including the codeunit summary).**

Test	What it proves
ManualModifyIsLogged	A manual modify of an exposed field on a registered table produces a Modify row.
ManualModifyLogsOncePerField	Exactly one row per changed field; no duplicates from the database-trigger events.
ManualDeleteIsLogged	A manual delete produces a Delete row.
ManualRenameIsLogged	A manual rename produces a Rename row for the key field (API field no_) with the old and new key values.
DateTimeLoggedAsUtcIso	Old and new values of a DateTime field equal Format(DT, 0, 9) and end in Z.
ZZDeleteApiCreatedDayPlanning	A Day Planning created through the API POST has no log row (inserts are never logged); deleted afterwards in a non-API session it produces exactly one Delete row.
ZZPurgeLeftovers	Helper: removes test data left in the log and the tables.

The run reported 8 PASS lines: the seven procedures above plus the codeunit-level summary line.

11.3 Live HTTP evidence on NL_Test

Against NL_Test the log API and the DayPlannings API of BC365Optimizer/Planning/v1.0 were called over HTTP with an app-only (client credentials) token; the token is not part of this document.

- GET list of UpdateAndDeleteLogs: succeeded (HTTP 200).
- POST, PATCH and DELETE of a Day Planning through the DayPlannings API succeeded (HTTP 200 / 204) and created **no** log rows: the latest entry number was unchanged after the calls. This is the evidence that changes made through the API are not logged (section 5.1).

11.4 What the evidence does not cover

> **Honest limit of the tests**
>
> The third-party skip itself (IsThirdPartyCall) cannot be unit-tested: AL tests do not run under the API client type, so CurrentClientType() is never Api / ODataV4 / SOAP inside a test. The skip is verified only by the live HTTP calls in 11.3. The remaining open points (non-admin users, the PATCH acknowledgement round trip, nested modifies) are listed in section 12.

12. Limitations, risks and open points

#	Topic	Detail	Impact / suggestion
1	API verified over HTTP only in part	The log API (GET list) and the DayPlannings API (POST, PATCH, DELETE) were called over HTTP on NL_Test with an app-only token (section 11.3). The PATCH of status on the log API and the full synchronisation loop against a real target system were not recorded.	Run the calls in 8.4 end to end against NL_Test before go-live; check the PATCH round trip.
2	Inserts are not logged	The Insert trigger is deliberately off. A record created in BC reaches the log only through later Modify rows (blank oldvalue).	The 3rd party must treat a Modify for an unknown systemId as 'create'. If a clean Create signal is needed, add an Insert action.
3	Cascade deletes of child records	Deleting a Job removes its Job Tasks, but only the Job Delete was logged; no Delete row for the tasks was produced.	A 3rd party mirroring Job Tasks keeps tasks of deleted jobs; it should delete children when it receives a parent Delete. Not investigated further.
4	No-trigger writes	Modify and Delete are logged through the database triggers, which fire regardless of the RunTrigger flag, so Modify(false) and Delete(false) are logged too. Rename depends on the global trigger.	Behaviour of Rename(false) not tested. Bulk jobs that use no-trigger writes on registered tables will produce log rows.
5	Session cache of the trigger setup	A new or removed setup row takes effect for new sessions only.	After changing the setup restart user sessions (or wait for them to end).
6	Non-admin users untested	All tests ran with an administrator-level user. InherentPermissions cover the app tables only.	Test with a limited user in a sandbox (do not change the permissions of real users to do so).
7	Value length	Old/New Value are Text[100]; longer values are truncated. API Field Name is Text[150].	Use the live record (step 6 in section 9) for long texts.
8	No purge	Synchronized rows are never removed.	Define a retention job (for example delete Synchronized rows older than N days).
9	Status casing	BC returns Pending / Synchronized.	Consumers should compare case-insensitively.
10	API page source filters are not applied	A registered API page may filter its source table (the Resource Skill page 50672 has SourceTableView Type = Resource). The log does not apply that filter, so changes to rows outside it (for example Resource Skill rows of another type) are logged as well.	3rd parties should ignore rows that do not belong to their scope, or the filter can be added to the log code.
11	Performance on busy tables	Each Modify loops all fields and does a metadata lookup per changed field and registered page. The database Modify trigger also adds one extra read of the record (the old-record capture).	Fine for master data; measure before registering high-volume tables (Sales Header/Line, ledger entries).
12	Changes made by the 3rd party are not logged	Modify, delete and rename through the API, OData or SOAP endpoints are skipped on purpose (no echo-back). The skip is per session client type, so AL cascades or side effects that run during an API request (a cascade delete, a recalculated field, a related record changed by a trigger) are not logged either.	The 3rd party must re-read dependent records after its own API call. Not a defect; the price of the no-echo rule.

#	Topic	Detail	Impact / suggestion
13	Single old-record slot	The old record is held in one slot per session. If another record is modified inside a Modify trigger of the first, the outer change is skipped instead of being logged with wrong old values (section 5.2).	Rare. A missing row is preferred over a wrong one. A per-record stack would remove the limit if it ever matters.
14	Old rows keep local-time date-times	Date-time values are logged as ISO 8601 UTC since 2026-10-20. Rows written before keep their old local-time text.	Do not mix old and new rows when comparing date-time old/new values.

13. Operations: adding a table, deployment, repairs, pitfalls

13.1 Track another table

- Make sure an API page exists for the table and that it exposes the fields the 3rd party needs (every exposed field is tracked).
- Open API Record Exposed Opti (page 50639), add a row, choose the API page in API Page No.; Table ID is filled automatically.
- Ask users to start a new session (trigger setup cache).
- Change a record and check page 50726 (Update and Delete Log List) for a Pending row with the expected API Field Name.

13.2 Build and deploy to the sandbox

- Compile with the AL build (app version stays 28.0.0.13; schema additions such as new fields synchronise without a version change in the sandbox).
- The MCP al_publish tool failed for NL_Test with 'Invalid URI'; the command-line tool works: `al publishapp <app> --project . --environmentname NL_Test --environmenttype Sandbox --tenant <tenantId>`.
- After publishing, open a new BC web session before testing triggers.

13.3 Repairing existing log rows

When the log format changed during development, existing rows were repaired with a temporary one-off procedure in report 50600 RepairData: edit the report, publish, run the report from the web client, then restore the report to its original content and republish so the repair cannot run again by accident. This pattern was used to clear test rows, strip an entity prefix from API Field Name, normalise old Rename rows and split a composite-key Rename into per-field rows.

13.4 Test checklist

Check	Expected
Change an exposed field (for example Resource Name) in the web client	One Modify row: API Field Name = name, old and new value, Status Pending
Change a non-exposed field (for example Resource Unit Price)	No row
Change a DateTime field of a registered table	Old/New Value in ISO 8601 UTC, ending in Z
Rename a single-key record	One Rename row, key field name and plain values

Check	Expected
Rename a Job Task (composite key)	Two Rename rows: jobNo_ and jobTaskNo_
Delete a record	One Delete row, blank field and values
Modify, delete or rename the record through the API / OData	No row (third-party changes are not logged)
Create a record through the API POST, or in the web client	No Insert row (inserts are never logged)
Table not in the setup	No row, no trigger
PATCH status = Synchronized on an entry	date_time_sync filled; back to Pending clears it

13.5 Pitfall: the global Modify and Delete events never fire in BC 28.5

The first implementation used the global trigger events OnAfterOnGlobalModify and OnAfterOnGlobalDelete (raised by GlobalTriggerManagement when the trigger mask bits 2 and 4 are set). Rename logging worked, but Modify and Delete logging produced no rows although the mask was set correctly and the setup row existed. Debugging showed that in BC 28.5 **only OnAfterOnGlobalRename fires**; the global Modify and Delete events are never raised.

> **Fix**
> Modify and Delete now use the database trigger events: OnAfterGetDatabaseTableTriggerSetup enables OnDatabaseModify and OnDatabaseDelete for registered tables; OnBeforeOnDatabaseModify captures the old record (the after event has no xRec); OnAfterOnDatabaseModify and OnAfterOnDatabaseDelete write the log. Rename stays on OnAfterGetGlobalTableTriggerMask + OnAfterOnGlobalRename. A side effect is that the database triggers fire regardless of the RunTrigger flag. The Update and Delete Log Action enum is unchanged: Modify, Delete, Rename (no Insert).

Lesson for similar work: do not trust an event just because it is declared in GlobalTriggerManagement. Prove that each event fires (a one-row test per action, as in section 11.2) before building on it, and remember that the after-modify database event does not provide the old record.

14. Appendix: source files

Object	File
Table 50614 API Record Exposed Opti	src/table/table_50614_APIRecordExposed.al
Page 50639 API Record Exposed Opti	src/page/page_50639_APIRecordExposed.al
Table 50607 Update and Delete Log Opti	src/table/table_50607_APIUpdateAndDeleteLog.al
Page 50726 Update and Delete Log List	src/page/page_50726_UpdateAndDeleteLogList.al
Enum 50624 Update and Delete Log Status	src/enum/enum_50624_UpdateAndDeleteLogStatus.al
Enum 50625 Update and Delete Log Action	src/enum/enum_50625_UpdateAndDeleteLogAction.al
Codeunit 50627 API Record Exposed Triggers	src/codeunit/codeunit_50627_APIRecordExposedTriggers.al
Codeunit 50605 API Record Exposed Mgt.	src/codeunit/codeunit_50605_TrustedCircle.al
API page 50727 UpdateAndDeleteLogApi Opt	src/api/page_50727_UpdateAndDeleteLog.api.al
Report 50600 RepairData (repair pattern)	src/report/report_50600_RepairDayPlanningResourceGroup.al
Codeunit 50900 API Rec Exposed Log Test	demolition_tests/ (test app DailyOptimizerTests)